

# Lab Program

## 1. Missionaries and Cannibals Problem

### Aim

To solve the Missionaries and Cannibals problem using Python.

### Algorithm

1. Start with 3 missionaries and 3 cannibals on the left side.
2. The boat carries 1 or 2 people.
3. Never allow cannibals to outnumber missionaries where missionaries are present.
4. Move people safely until everyone reaches the right side.

### Pseudocode

Start

Initial state = (3,3,1)

Goal state = (0,0,0)

While goal is not reached:

    Generate possible boat moves

    Check safe state

    Add new state

Print solution

Stop

### Code

```
from collections import deque
start = (3, 3, 1)
goal = (0, 0, 0)
moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
queue = deque([start])
visited = {start}
while queue:
    m, c, b = queue.popleft()
```

```

print((m, c, b))

if (m, c, b) == goal:
    break

for x, y in moves:
    if b == 1:
        new = (m-x, c-y, 0)
    else:
        new = (m+x, c+y, 1)
    nm, nc, nb = new
    if 0 <= nm <= 3 and 0 <= nc <= 3:
        if (nm == 0 or nm >= nc) and (3-nm == 0 or 3-nm >= 3-nc):
            if new not in visited:
                visited.add(new)
                queue.append(new)

```

output :

```

C:\Users\B DEEPHI\Downloads\AI programme> python missionaries_cannibals.py
43 solution = solve()
44 if solution:
45     print_path(solution)
46 else:
47     print("No solution found.")

PS C:\Users\B DEEPHI\Downloads\AI programme> & 'C:\Users\B DEEPHI\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\B DEEPHI\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' "51804" "-." 'C:\Users\B DEEPHI\Downloads\AI programme\missionaries_cannibals.py'

Step 0: Left - M:3 C:3 | Right - M:0 C:0 | Boat on left
Step 1: Left - M:3 C:1 | Right - M:0 C:2 | Boat on right
Step 2: Left - M:3 C:2 | Right - M:0 C:1 | Boat on left
Step 3: Left - M:3 C:0 | Right - M:0 C:3 | Boat on right
Step 4: Left - M:3 C:1 | Right - M:0 C:2 | Boat on left
Step 5: Left - M:1 C:1 | Right - M:2 C:2 | Boat on right
Step 6: Left - M:2 C:2 | Right - M:1 C:1 | Boat on left
Step 7: Left - M:0 C:2 | Right - M:3 C:1 | Boat on right
Step 8: Left - M:0 C:3 | Right - M:3 C:0 | Boat on left
Step 9: Left - M:0 C:1 | Right - M:3 C:2 | Boat on right
Step 10: Left - M:0 C:2 | Right - M:3 C:1 | Boat on left
Step 11: Left - M:0 C:0 | Right - M:3 C:3 | Boat on right

PS C:\Users\B DEEPHI\Downloads\AI programme>

```

Result:

The Program Missionaries and Cannibals Problem was executed successfully

## 2. Vacuum Cleaner Problem

### Aim

To solve the Vacuum Cleaner problem using Python.

### Algorithm

1. Start with two rooms.
2. Check the current room.
3. If the room is dirty, clean it.
4. Move to the next room.
5. Repeat until all rooms are clean.

### Pseudocode

Start

Set rooms as Dirty

For each room:

    If room is Dirty:

        Clean the room

Print final state

Stop

### Code

```
def vacuum_cleaner(location, state):  
    print("Vacuum is placed in Room", location)  
    print("Room A:", state[0], " | Room B:", state[1])  
    if location == "A":  
        if state[0] == "Dirty":  
            print("Room A is Dirty. Cleaning Room A...")  
            state[0] = "Clean"  
        else:  
            print("Room A is already Clean.")  
    location = "B"  
    if state[1] == "Dirty":  
        print("Room B is Dirty. Cleaning Room B...")
```

```

        state[1] = "Clean"

    else:

        print("Room B is already Clean.")

    elif location == "B":

        if state[1] == "Dirty":

            print("Room B is Dirty. Cleaning Room B...")

            state[1] = "Clean"

        else:

            print("Room B is already Clean.")

    location = "A"

    if state[0] == "Dirty":

        print("Room A is Dirty. Cleaning Room A...")

        state[0] = "Clean"

    else:

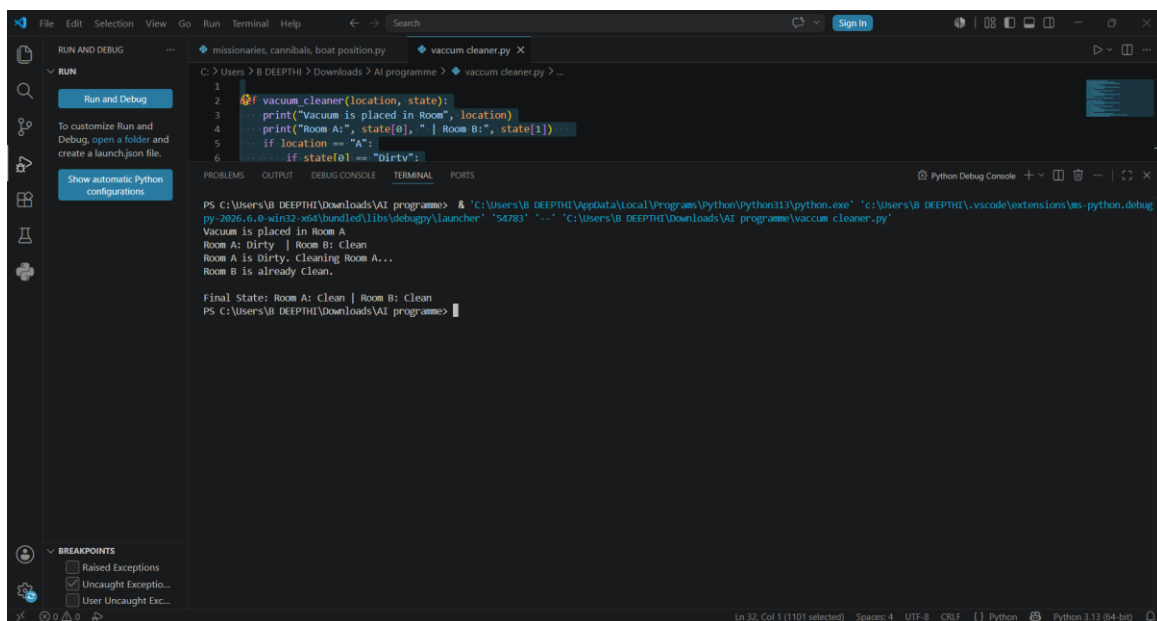
        print("Room A is already Clean.")

    print("\nFinal State: Room A:", state[0], "| Room B:", state[1])

vacuum_cleaner("A", ["Dirty", "Clean"])

```

## output



The screenshot shows a VS Code editor with a Python file named `vacuum_cleaner.py` open. The code is a recursive function that cleans rooms A and B based on their state (Dirty or Clean). The terminal output shows the execution of the function, starting with 'Vacuum is placed in Room A', followed by cleaning Room A and then Room B, and finally printing the final state: 'Final State: Room A: Clean | Room B: Clean'.

```

1  def vacuum_cleaner(location, state):
2      print("Vacuum is placed in Room", location)
3      print("Room A:", state[0], " | Room B:", state[1])
4      if location == "A":
5          if state[0] == "Dirty":
6              print("Room A is Dirty. Cleaning Room A...")
7              state[0] = "Clean"
8          else:
9              print("Room A is already Clean.")
10         if location == "B":
11             if state[1] == "Dirty":
12                 print("Room B is Dirty. Cleaning Room B...")
13                 state[1] = "Clean"
14             else:
15                 print("Room B is already Clean.")
16         location = "A"
17         if state[0] == "Dirty":
18             print("Room A is Dirty. Cleaning Room A...")
19             state[0] = "Clean"
20         else:
21             print("Room A is already Clean.")
22         print("\nFinal State: Room A:", state[0], " | Room B:", state[1])
23     vacuum_cleaner("A", ["Dirty", "Clean"])

```

```

PS C:\Users\B DEEPTI\Downloads\AI programme> & 'C:\Users\B DEEPTI\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\B DEEPTI\.vscode\extensions\ms-python.debugpy-2026.6.8-win32-x64\bundle\libs\debugpy\launcher' '54783' '-' 'C:\Users\B DEEPTI\Downloads\AI programme\vacuum_cleaner.py'
Vacuum is placed in Room A
Room A: Dirty | Room B: Clean
Room A is Dirty. Cleaning Room A...
Room B is already Clean.

Final State: Room A: Clean | Room B: Clean
PS C:\Users\B DEEPTI\Downloads\AI programme>

```